# 12. Jobs, Background, nohup, screen/tmux intro

## 12. Jobs, Background, nohup, screen/tmux intro

**Phase:** Phase 4 — Processes and jobs
**Estimated time:** 1 day
**Slug:** `12-jobs-background`

### Learning objectives

- Run jobs in background
- Use fg/bg/jobs
- Keep sessions with tmux
- Apply nohup carefully

### Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

### Why this matters

A command that takes minutes should not freeze your terminal, and a long job over SSH should not die when the connection drops. Shell job control lets you run work in the background and bring it back; `tmux` goes further by keeping whole terminal sessions alive on the server so you can detach and reattach later from anywhere.

### Core concepts

The shell tracks *jobs*: pipelines started from this shell, each with a job number (`%1`). A job runs in the *foreground* (owns the terminal) or *background*. Add `&` to start in the background; press Ctrl-Z to suspend the foreground job (SIGTSTP), then `bg` resumes it in the background or `fg` brings it forward.

| Command | Effect |
|---|---|
| `cmd &` | Start in background |
| `jobs` | List this shell's jobs |
| `fg %N` | Bring job N to foreground |
| `bg %N` | Continue stopped job in background |
| Ctrl-Z | Suspend foreground job |
| Ctrl-C | Interrupt foreground job (SIGINT) |
| `wait` | Block until background jobs finish |
| `nohup cmd &` | Ignore SIGHUP, output goes to `nohup.out` |
| `disown %N` | Remove job from shell's table so HUP is not sent |

When a terminal closes, the shell sends SIGHUP to its jobs and they usually die. `nohup` and `disown` mitigate that, but `tmux` (or `screen`) is more robust: the session lives in a server process independent of your terminal.

```
  your terminal  <--- attach / detach --->  [ tmux server ]
                                             |-- session "work"
                                             |     |-- window 0: shell, vim
                                             |     +-- window 1: logs (pane
split)
                                             +-- session "build"
   SSH drops -> tmux server and programs keep running
```

tmux commands begin with the prefix, Ctrl-b by default, followed by a key.

| Keys / command | Action |
|---|---|
| `tmux new -s NAME` | New named session |
| Ctrl-b d | Detach from session |
| `tmux ls` | List sessions |
| `tmux attach -t NAME` | Reattach |
| Ctrl-b c | New window |
| Ctrl-b n / p | Next / previous window |
| Ctrl-b % | Split into left/right panes |
| Ctrl-b ” | Split into top/bottom panes |
| Ctrl-b [ | Scroll/copy mode (q to exit) |
| `tmux kill-session -t NAME` | Remove session |

## Worked examples

Background and job control:

```
sleep 120 &
# sample output: [1] 5012
jobs
# sample output: [1]+  Running                 sleep 120 &
fg %1
# sample output: sleep 120
# (press Ctrl-Z)
# sample output: [1]+  Stopped                 sleep 120
bg %1
# sample output: [1]+ sleep 120 &
```

Wait for background work:

```
(sleep 2; echo A) & (sleep 1; echo B) & wait
# sample output: B
# sample output: A
```

Survive logout:

```
nohup ./long_task.sh > task.log 2>&1 &
# sample output: [1] 5120
disown -h %1
```

tmux workflow:

```
tmux new -s build
# sample output: (new green status bar at the bottom)
# run: make -j4   then press Ctrl-b d
# sample output: [detached (from session build)]
tmux ls
# sample output: build: 1 windows (created Wed Oct 21 10:10:00 2026)
tmux attach -t build
```

Scripted detached session:

```
tmux new -d -s logs 'tail -f /var/log/syslog'
tmux send-keys -t logs C-c
tmux kill-session -t logs
```

## Mini-lab

1. Install tmux: `sudo apt update && sudo apt install -y tmux` (or in Docker `apt install -y tmux`).
2. Run `sleep 300 &`, then `jobs`, `fg`, Ctrl-Z, `bg`, and finally `kill %1`.
3. Run `tmux new -s lab12`, then `top` in it.
4. Press Ctrl-b d; confirm your prompt returned; run `tmux ls`.
5. Reattach with `tmux attach -t lab12`; top is still running; quit with `q`.
6. Split the window with Ctrl-b % and run `date` on one side; move panes with Ctrl-b arrow.

7. Create a window with Ctrl-b c and switch with Ctrl-b n and p.
8. Exit all shells (`exit`) or `tmux kill-server` to clean up.

## Decision guide / failure modes

Use `&` and `jobs` for short tasks in one shell; use `nohup` for a single fire-and-forget command; use `tmux` for interactive long-running work, multiple panes or remote sessions; use systemd services or cron for things that must start at boot or run unattended.

Common failures:

- Closing the terminal kills plain background jobs because of SIGHUP.
- Background job still writes to the terminal and interrupts typing; redirect its output.
- Background jobs reading stdin get stopped (SIGTTIN); redirect `< /dev/null`.
- Starting tmux inside tmux creates nesting and confusing prefixes; check `$TMUX`.
- Forgetting the session name, then `tmux ls` to find it; many accidental sessions waste memory.
- Job numbers (`%1`) are per shell and are not PIDs.

## Common mistakes

- Leaving critical prod tasks only in a laptop terminal
- nohup without log rotation
- Detaching mentally but not from tmux

## Exercises

Work in `exercises/12-jobs-background/`. Read that folder's `README.md` fully before coding.

```
cd exercises/12-jobs-background
chmod +x test.sh exercise.sh
./test.sh
```

### What you will build

Implement tmux_detach_keys(). It takes no arguments and returns the default key sequence used to detach from a tmux session, as the lowercase string "ctrl-b d". That means pressing Ctrl-b (the tmux prefix), releasing it, and then pressing d. The session and everything running in it continue in the background; you can come back later with "tmux attach -t NAME". Return exactly that text: lowercase, a hyphen in ctrl-b, and a single space before d. Do not return the tmux command line (like "tmux detach"), only the keystroke description.

### Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

**Do not open `solutions/12-jobs-background/` until you have an honest failing attempt.**

## Self-check

Try to answer without scrolling up:

1. How detach tmux?
2. What does & do?
3. When use systemd instead of nohup?

## Further reading

- https://github.com/tmux/tmux/wiki